

BUỔI 4:

CÁC HỆ MẬT MÃ BẤT ĐỐI XỨNG

Sinh viên hoàn thành nội dung thực. Nộp file trả lời (.pdf) và file code:

- File pdf: bao gồm trả lời câu hỏi.
- File code
- Lưu ý: sử dụng `Console.OutputEncoding = Encoding.UTF8` và `Console.InputEncoding = Encoding.UTF8`

để đọc và hiển thị tiếng Việt

1. RSA

Cần thực hiện các bước:

- Sinh khóa bí mật và công khai lưu lại file
- Thực hiện mã hóa thông điệp bằng khóa công khai
- Lưu bản mã vào file
- Giải mã bằng khóa bí mật

Hàm sinh khóa:

```
void GenerateAndSaveRsaKeys(string RsaPublicKeyFile, string RsaPrivateKeyFile)
{
    try
    {
        using RSA rsa = RSA.Create(2048);
        string publicKeyPem = rsa.ExportSubjectPublicKeyInfoPem();
        string privateKeyPem = rsa.ExportPkcs8PrivateKeyPem();
        File.WriteAllText(
            RsaPublicKeyFile,
            publicKeyPem,
            Encoding.UTF8);
        File.WriteAllText(
            RsaPrivateKeyFile,
            privateKeyPem,
            Encoding.UTF8);
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Không thể tạo khóa RSA: {ex.Message}");
    }
}
```

Hàm mã hóa

```
void EncryptWithRsaPublicKey(string RsaPublicKeyFile, string RsaCiphertextFile)
{
    if (!File.Exists(RsaPublicKeyFile))
```

```

    {
        Console.WriteLine("Chưa tìm thấy khóa công khai RSA.");
        return;
    }
    Console.WriteLine("Nhập nội dung cần mã hóa: ");
    string plaintext = Console.ReadLine() ?? "";

    try
    {
        string publicKeyPem = File.ReadAllText(RsaPublicKeyFile, Encoding.UTF8);
        using RSA rsa = RSA.Create();
        rsa.ImportFromPem(publicKeyPem);
        byte[] plaintextBytes = Encoding.UTF8.GetBytes(plaintext);
        byte[] ciphertextBytes = rsa.Encrypt(plaintextBytes, RSAEncryptionPadding.OaepSHA256);
        string ciphertextBase64 = Convert.ToBase64String(ciphertextBytes);
        File.WriteAllText(RsaCiphertextFile, ciphertextBase64, Encoding.UTF8);
    }
    catch (CryptographicException)
    {
        Console.WriteLine("Không thể mã hóa RSA.");
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Lỗi: {ex.Message}");
    }
}

```

Hàm giải mã

```

void DecryptWithRsaPrivateKey(string RsaPrivateKeyFile, string RsaCiphertextFile)
{
    if (!File.Exists(RsaPrivateKeyFile))
    {
        Console.WriteLine("Chưa tìm thấy khóa bí mật RSA.");
        return;
    }
    if (!File.Exists(RsaCiphertextFile))
    {
        Console.WriteLine("Chưa tìm thấy bản mã RSA.");
        return;
    }
    try
    {
        string privateKeyPem = File.ReadAllText(RsaPrivateKeyFile, Encoding.UTF8);
        string ciphertextBase64 = File.ReadAllText(RsaCiphertextFile, Encoding.UTF8).Trim();
        byte[] ciphertextBytes = Convert.FromBase64String(ciphertextBase64);
        using RSA rsa = RSA.Create();
        rsa.ImportFromPem(privateKeyPem);
        byte[] plaintextBytes = rsa.Decrypt(ciphertextBytes, RSAEncryptionPadding.OaepSHA256);
        string plaintext = Encoding.UTF8.GetString(plaintextBytes);
        Console.WriteLine("Nội dung sau khi giải mã:");
        Console.WriteLine(plaintext);
    }
    catch (FormatException)
    {
        Console.WriteLine("Bản mã không đúng định dạng Base64.");
    }
    catch (CryptographicException)
    {
        Console.WriteLine("Không thể giải mã RSA.");
        Console.WriteLine("Khóa bí mật có thể không thuộc cặp khóa đã mã hóa.");
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Lỗi: {ex.Message}");
    }
}

```

Thực hiện lại mã hóa và giải mã RSA.

Tìm hiểu các lớp + hàm, cho biết lớp/hàm có ý nghĩa gì? Các thuộc tính có đặc điểm gì? Với hàm, các tham số quan trọng có ý nghĩa gì?

- Class RSA.
- RSAEncryptionPadding.OaepSHA256
- Hàm ExportSubjectPublicKeyInfoPem, ExportPkcs8PrivateKeyPem, ImportFromPem

2. ELGAMAL

Thêm BouncyCastle vào project (sử dụng NuGet)

Thực hiện đoạn chương trình

```
string plaintext = "Hello CTUT";
byte[] plainBytes = Encoding.UTF8.GetBytes(plaintext);
var random = new SecureRandom();
var paramGen = new ElGamalParametersGenerator();
paramGen.Init(256, 10, random);
ElGamalParameters parameters = paramGen.GenerateParameters();
var keyGen = new ElGamalKeyPairGenerator();
keyGen.Init(new ElGamalKeyGenerationParameters(random, parameters));
AsymmetricCipherKeyPair keyPair = keyGen.GenerateKeyPair();
var encryptEngine = new ElGamalEngine();
encryptEngine.Init(true, keyPair.Public);
byte[] cipherBytes = encryptEngine.ProcessBlock(plainBytes, 0, plainBytes.Length);
Console.WriteLine($"Ciphertext (Base64): {Convert.ToBase64String(cipherBytes)}");
var decryptEngine = new ElGamalEngine();
decryptEngine.Init(false, keyPair.Private);
byte[] decryptedBytes = decryptEngine.ProcessBlock(cipherBytes, 0, cipherBytes.Length);
string decryptedText = Encoding.UTF8.GetString(decryptedBytes);
Console.WriteLine($"Decrypted: {decryptedText}");
```

Giải thích lại các hàm chính đã sử dụng trong quá trình mã hóa và giải mã.

Điều chỉnh đoạn code để thực hiện quy trình tương tự phần 1.

- Sinh khóa và lưu lại trong file
- Sử dụng khóa công khai lưu trong file để mã hóa, lưu lại bản mã trong file
- Sử dụng khóa bí mật lưu trong file và bản mã đã lưu để giải mã